

Reporte de Debug del Sitio Web

Diagnóstico completo del repositorio **Pagina-Arquidiocesis**: seguridad, funcionalidad, contenido, SEO y despliegue. Hallazgos priorizados con recomendaciones accionables.

Resumen ejecutivo

El sitio es un portal institucional estático (HTML + Tailwind) con funciones dinámicas servidas por funciones *serverless* en Vercel, una base de datos en Supabase y un Portal de Sacerdotes con generación de documentos por IA. La estructura general es sólida: las 13 páginas públicas tienen idioma declarado, *meta description* y *viewport*; las 130 imágenes tienen texto alternativo; y las claves de API se manejan correctamente con variables de entorno. Sin embargo, la auditoría detectó **un fallo de seguridad crítico** que compromete por completo el modelo de acceso del portal, además de imágenes rotas, contenido incompleto y varios puntos de SEO y despliegue por resolver.

Severidad	Hallazgos	Naturaleza
CRÍTICA	3	Seguridad y autenticación
ALTA	4	Funcionalidad y contenido visible
MEDIA	6	SEO, despliegue, mantenibilidad
BAJA	3	Mejoras y robustez

Acción inmediata: retirar del repositorio y del sitio el archivo `parishes_with_passwords.json` y rotar todas las claves de parroquia. Hasta entonces, cualquier persona puede iniciar sesión como cualquier parroquia.

PRIORIDAD 1 · SEGURIDAD

Hallazgos críticos

1. Contraseñas de parroquias expuestas públicamente	CRÍTICA
El archivo <code>parishes_with_passwords.json</code> contiene 80 parroquias con su contraseña en texto plano (claves numéricas de 5 dígitos, p. ej. «80606») junto a datos personales de los párrocos (correos y teléfonos). El archivo está versionado en un repositorio público y, además, se sirve como archivo estático: se carga desde <code>portal-sacerdotes.html</code>, <code>validar.html</code> y <code>code.html</code>. Impacto: cualquier persona puede descargar el archivo e iniciar sesión como cualquier parroquia, presentar trámites en su nombre y acceder a datos personales del clero. El sistema de acceso del portal queda anulado. Recomendación: <i>Eliminar el archivo del repositorio y del sitio; mover la autenticación al servidor (Supabase Auth o función <i>serverless</i> que valide contra la base de datos); almacenar solo hashes de contraseña; sustituir las claves de 5 dígitos por credenciales más robustas y rotarlas todas. Recordar que borrarlo del repo no basta: queda en el historial de Git, por lo que también debe purgarse del historial.</i>	
2. Autenticación validada en el navegador (cliente)	CRÍTICA
El inicio de sesión de parroquias y sacerdotes se resuelve en el lado del cliente comparando lo que escribe el usuario contra un JSON descargado por el navegador. No existe una verificación en el servidor. Impacto: cualquier usuario con conocimientos básicos puede saltarse el login manipulando el código en el navegador, sin necesidad siquiera de conocer la contraseña. Recomendación: <i>Implementar autenticación del lado del servidor con emisión de sesión/token (Supabase Auth o JWT). El cliente nunca debe poseer la lista completa de credenciales ni decidir por sí mismo si el acceso es válido.</i>	

3. CORS totalmente abierto en las APIs

CRÍTICA

Las cuatro funciones serverless (/api/config, generate-document, send-email, instagram) responden con Access-Control-Allow-Origin: *, permitiendo que cualquier sitio externo las invoque.

Impacto: otros sitios pueden consumir los endpoints de envío de correo y generación de documentos, abriendo la puerta a abuso, spam o costos no deseados en los servicios de IA y correo.

Recomendación: Restringir el origen permitido al dominio oficial (pagina-arquidiocesis.vercel.app) y añadir validación/limitación de tasa en las funciones que envían correo o consumen IA.

PRIORIDAD 2 · ALTA

Funcionalidad y contenido

4. Imágenes de cabecera rotas (extensión incorrecta)

ALTA

Varias páginas piden imágenes que no existen porque se referencian con extensión .webp cuando el archivo no está presente: img/ambientes.webp, img/pastoral.webp, img/mapa_arquidiocesis.webp y img/catedral.webp (existe catedral.jpg, no el .webp).

Impacto: las cabeceras de esas secciones caen al *placeholder* de Unsplash (una foto genérica de una iglesia que no corresponde), dando aspecto de error y restando profesionalidad.

Recomendación: Generar y subir las imágenes en .webp con esos nombres exactos, o corregir las rutas para que apunten a los archivos existentes (.jpg/.png).

5. Cobertura de fotos de parroquias muy baja

ALTA

El directorio muestra una foto por parroquia (img/templos/\${code}.webp), pero solo existen **28 fotos para 72 parroquias**. El archivo Fotos_necesarias.txt, pensado para listar las que faltan, está vacío.

Impacto: el 61% de las parroquias (44) muestra un *placeholder* genérico en lugar de su imagen real, lo que da una impresión de directorio incompleto.

Recomendación: Recopilar y optimizar las fotos faltantes; mientras tanto, usar un *placeholder* propio con la identidad de la Arquidiócesis (no uno externo) y completar el inventario de fotos pendientes.

6. Referencias a fotos inexistentes en static_cards.html

ALTA

static_cards.html referencia decenas de imágenes de las zonas PZ4 a PZ7 (img/templos/PZ4-1.webp...), pero la carpeta solo contiene fotos de PZ1 a PZ3.

Impacto: si esta página se publica, mostrará numerosas imágenes rotas o placeholders.

Recomendación: Confirmar si static_cards.html debe formar parte del sitio; si es un archivo de trabajo, excluirlo del despliegue; si es definitivo, completar las imágenes.

7. Inconsistencia entre fuentes de datos de parroquias

ALTA

Coexisten tres listados: `parishes.json` en la raíz (72 registros), una copia idéntica en `js/parishes.json` (72) y `parishes_with_passwords.json` (80). Las cifras no coinciden.

Impacto: riesgo de que el directorio, el portal y el sistema de login muestren datos distintos según qué archivo lea cada página; mantenimiento duplicado y propenso a errores.

Recomendación: Definir una única fuente de verdad (idealmente la tabla en Supabase) y eliminar las copias redundantes. Conciliar la diferencia de 8 registros.

PRIORIDAD 3 · MEDIA

SEO, despliegue y mantenibilidad

8. Sin Open Graph ni Twitter Cards

MEDIA

Ninguna de las 13 páginas incluye etiquetas `og:` ni *Twitter Cards*.

Impacto: al compartir cualquier enlace en WhatsApp, Facebook o Instagram no aparece vista previa (título, descripción ni imagen), lo que reduce muchísimo el alcance al difundir el sitio.

Recomendación: Añadir `og:title`, `og:description`, `og:image` y `og:url` en cada página, con una imagen institucional de 1200×630 px.

9. Faltan favicon, robots.txt, sitemap.xml y página 404

MEDIA

No existen `favicon.ico`, `robots.txt`, `sitemap.xml`, `404.html` ni `manifest.json`.

Impacto: sin favicon la pestaña del navegador se ve genérica; sin `sitemap/robots` el posicionamiento en Google es más débil; sin 404 propia, las rutas erróneas muestran una página poco amable.

Recomendación: Agregar un favicon con el escudo, un `robots.txt` y un `sitemap.xml` con las URLs públicas, y una página 404 con la identidad del sitio. El `manifest.json` es opcional (para instalación como PWA).

10. Tailwind cargado por CDN en producción

MEDIA

Las 14 páginas usan `cdn.tailwindcss.com`. La propia documentación de Tailwind indica que el CDN no está pensado para producción.

Impacto: carga más lenta (se descarga todo el framework y se compila en el navegador), parpadeo de estilos sin conexión al CDN y dependencia de un servicio externo.

Recomendación: Compilar Tailwind en el build y servir un CSS propio ya purgado, con lo que el sitio carga solo los estilos que realmente usa.

11. Archivos de desarrollo desplegados en producción

MEDIA

El despliegue arrastra material de trabajo: `code.html` (6.119 líneas, duplicado del portal), `static_cards.html`, la carpeta `imagenes_originales/` (17 MB de originales sin comprimir), decenas de scripts `.py`, archivos `.txt` de trabajo y los PDF/DOCX fuente.

Impacto: se publica contenido que no es del sitio, aumentan el peso y el tiempo de despliegue, y se expone material interno.

Recomendación: Mover los scripts y fuentes a una carpeta de herramientas fuera del despliegue (o usar un `.vercelignore`), y comprimir o retirar `imagenes_originales/`.

12. Datos de parroquias duplicados en dos rutas

MEDIA

`parishes.json` existe tanto en la raíz como en `js/`.

Impacto: cualquier actualización debe hacerse en dos sitios; si solo se cambia uno, las páginas se desincronizan.

Recomendación: *Conservar una sola copia (o generarla desde la base de datos) y apuntar todas las páginas a esa ruta.*

13. Mensajes de depuración en el código publicado

MEDIA

Hay 105 llamadas `console.log/warn/error` en el código que llega al navegador.

Impacto: ruido en la consola, posible filtración de detalles internos y un acabado menos profesional.

Recomendación: *Retirar los `console.log` de depuración (o eliminarlos automáticamente en el build), conservando solo los estrictamente necesarios.*

PRIORIDAD 4 · BAJA

Mejoras y robustez

14. Páginas internas poco descubribles

BAJA

`portal-sacerdotes.html` y `validar.html` no están enlazadas desde el inicio; solo se llega a ellas a través de `cancilleria.html`.

Impacto: menor visibilidad de funciones clave para quien no conoce la ruta exacta.

Recomendación: *Si se desea, añadir accesos directos contextualizados (p. ej. un enlace discreto al portal y al verificador de documentos desde el pie de página).*

15. Dependencia de servicios externos para funciones núcleo

BAJA

El sitio depende de `api.qrserver.com` (códigos QR), `placeholder.co` (placeholders), la Instagram Graph API (cuyo token caduca cada 60 días) y Resend (correo).

Impacto: si uno de esos servicios falla o el token de Instagram expira, se rompen funciones visibles (p. ej. el feed de noticias dejará de cargar al vencer el token).

Recomendación: *Documentar la renovación del token de Instagram (recordatorio cada 60 días), y valorar generar los QR del lado del servidor y alojar un placeholder propio para no depender de terceros.*

16. Endurecer cabeceras de seguridad

BAJA

No se observan cabeceras de seguridad (CSP, X-Content-Type-Options, Referrer-Policy, etc.) en `vercel.json`.

Impacto: menor protección frente a inyección de contenido y otras técnicas; nada explotable de inmediato, pero es una buena práctica pendiente.

Recomendación: *Añadir cabeceras de seguridad básicas en `vercel.json` una vez resueltos los puntos críticos.*

BALANCE

Lo que ya está bien resuelto

✓ Las 13 páginas públicas declaran idioma, *meta description* y *viewport* (base de SEO y accesibilidad correcta).

- ✓ Las 130 etiquetas incluyen texto alternativo (alt).
- ✓ Las claves de API (Supabase, Groq, Resend, Instagram) se manejan con variables de entorno; no hay secretos escritos en el código de las funciones.
- ✓ El .gitignore protege correctamente los archivos .env.
- ✓ Las imágenes del directorio tienen un *fallback* (onerror) que evita el icono de imagen rota.
- ✓ El generador de documentos por IA incluye plantillas de respaldo si el servicio de IA no está disponible.
- ✓ No se detectaron enlaces internos rotos entre páginas ni desequilibrios de etiquetas en el HTML revisado.

CÓMO AVANZAR

Plan de acción sugerido

Fase	Qué resolver	Hallazgos
Ahora	Retirar el JSON de contraseñas, mover el login al servidor y cerrar el CORS.	1, 2, 3
Esta semana	Arreglar imágenes de cabecera, unificar datos de parroquias y depurar archivos de desarrollo del despliegue.	4, 7, 11, 12
Corto plazo	Completar fotos de parroquias, añadir Open Graph, favicon, sitemap y 404.	5, 6, 8, 9
Mejora continua	Compilar Tailwind, limpiar consola, endurecer cabeceras y reducir dependencias externas.	10, 13, 15, 16

Nota de alcance: esta auditoría se basa en el análisis estático del código del repositorio (estructura, enlaces, referencias de recursos, configuración y funciones serverless). No incluye pruebas dinámicas en el navegador, pruebas de penetración ni medición de rendimiento en vivo. El hallazgo crítico de contraseñas conviene atenderlo cuanto antes.